جلسهٔ ۳ — فیلترِ مکانی و کانولوشن

جزوهٔ درس — هفتهٔ سوم

مکملِ اسلایدهای انگلیسیِ جلسهٔ ۳. تصاویر از دیتاستِ واقعیِ شما (پلاک و کنتور) تولید شده‌اند. هدف: عملیاتِ همسایگی — یک کرنل روی تصویر می‌لغزد و هر پیکسل را با همسایه‌هایش ترکیب می‌کند.

فهرست مطالب

۱ از عملیاتِ نقطه‌ای به عملیاتِ همسایگی ۱

۲ کانولوشن ۱

۱.۲ مدیریتِ لبه و نرمال‌سازی ۲

۳ کانولوشن، گام‌به‌گام ۲

۴ هموارسازی (Smoothing) ۳

۱.۴ تاریِ جعبه‌ای در برابرِ گاوسی — روی پلاکِ واقعی ۳

۲.۴ چرا اصلاً هموار کنیم؟ ۳

۵ تیزسازی (Sharpening) ۴

۶ تمرینِ کلاسیِ وسط‌جلسه (۱۲ دقیقه) ۴

۷ کاربردهای فیلترِ مکانی ۵

۱.۷ جدولِ انتخابِ فیلتر ۵

۸ بخشِ عمیق ۵

۱.۸ دیدِ فرکانسی: پایین‌گذر در برابرِ بالاگذر ۵

۲.۸ تصویرِ انتگرالی: تاریِ جعبه‌ای در زمانِ ثابت ۶

۳.۸ اثراتِ مرزی با جزئیات ۶
۴.۸ تفکیک‌پذیری و سرعت ۶
۵.۸ فیلترِ غیرخطی: میانه (Median) ۶
۶.۸ فیلترِ خطی در برابرِ غیرخطی ۷
۷.۸ فیلترِ دوطرفه (Bilateral) ۷

۹ فیلترها در کد ۷

۱.۹ پارامترها و دام‌ها ۸

۱۰ باغِ کرنل‌ها و طراحیِ آن‌ها ۸

۱۱ کانولوشن در برابرِ همبستگی ۹

۱۲ این جلسه در پروژهٔ نهایی ۹

۱۳ جمع‌بندی ۹

۱ از عملیاتِ نقطه‌ای به عملیاتِ همسایگی

در جلسهٔ ۲ عملیاتِ نقطه‌ای فقط به مقدارِ یک پیکسل نگاه می‌کردند. آن‌ها نمی‌توانند تار یا تیز کنند، نویز را حذف کنند، یا لبه بیابند — چون همهٔ این‌ها به همسایه‌های یک پیکسل نیاز دارند. امروز **فیلترِ مکانی** را یاد می‌گیریم: یک **کرنل** کوچک روی تصویر می‌لغزد و هر پیکسل را با همسایه‌هایش ترکیب می‌کند. این عمل **کانولوشن** (convolution) است.

نکتهٔ کلیدی

کانولوشن دقیقاً همان عملیاتی است که یک شبکهٔ کانولوشنی (CNN) در جلسهٔ ۱۳ یاد می‌گیرد. تفاوت: اینجا مقادیرِ کرنل را ما طراحی می‌کنیم؛ در CNN شبکه آن‌ها را از داده می‌آموزد. هر چیزی که امروز دربارهٔ کرنل یاد بگیرید، همان کاری است که لایه‌های اولِ یک CNN می‌کنند.

۲ کانولوشن

یک کرنل (فیلتر/ماسک) روی تصویر می‌لغزد؛ برای هر پیکسل: همسایگی را در کرنل ضرب کنید، جمع بزنید، نتیجه را بنویسید. سپس بلغزید و تکرار کنید.

ریاضیات

$$g(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} h(i,j)\, f(x-i,\, y-j)$$

f = تصویر، h = کرنل، g = خروجی. کانولوشن کرنل را برمی‌گرداند؛ **همبستگی** (correlation) نه. برای کرنل‌های متقارن، این دو یکسان‌اند.

مقادیرِ کرنل، عملیات را تعریف می‌کنند: میانگین ← تاری؛ تفاضل ← لبه. دو مثال:

تیزسازی:
$$\begin{bmatrix} ۰ & -۱ & ۰ \\ -۱ & ۵ & -۱ \\ ۰ & -۱ & ۰ \end{bmatrix}$$

تاریِ جعبه‌ای:
$$\frac{۱}{۹}\begin{bmatrix} ۱ & ۱ & ۱ \\ ۱ & ۱ & ۱ \\ ۱ & ۱ & ۱ \end{bmatrix}$$

نکتهٔ کلیدی

کرنل همان الگوریتم است. اعداد را عوض کنید، اثر عوض می‌شود — هیچ چیزِ دیگری. این ساده‌ترین و قدرتمندترین ایدهٔ این جلسه است.

۱.۲ مدیریتِ لبه و نرمال‌سازی

در مرزِ تصویر، کرنل بیرون می‌زند — با صفر، تکرار، یا انعکاس پُر کنید (borderType). یک کرنلِ تاری باید **مجموعش ۱** باشد (حفظِ روشنایی)؛ یک کرنلِ لبه مجموعش ۰. کرنلِ نرمال‌نشده ← تصویر به‌عنوانِ اثرِ جانبی تاریک/روشن می‌شود.

هشدار / دام

فراموش‌کردنِ نرمال‌سازیِ یک کرنلِ تاری، دلیلِ آن است که یک تصویرِ «نرم‌شده» گاهی خیلی تاریک درمی‌آید. همیشه مجموعِ کرنلِ تاری را ۱ نگه دارید.

۳ کانولوشن، گام‌به‌گام

بیایید یک کانولوشن را کامل انجام دهیم. فرض کنید این وصله ۳ × ۳ از تصویر و این کرنلِ تاریِ جعبه‌ای را داریم:

$$P = \begin{bmatrix} ۱۰۰ & ۱۰۰ & ۱۰۰ \\ ۱۰۰ & ۲۰۰ & ۱۰۰ \\ ۱۰۰ & ۱۰۰ & ۱۰۰ \end{bmatrix} \qquad H = \frac{۱}{۹}\begin{bmatrix} ۱ & ۱ & ۱ \\ ۱ & ۱ & ۱ \\ ۱ & ۱ & ۱ \end{bmatrix}$$

خروجیِ پیکسلِ مرکزی = میانگینِ همهٔ ۹ مقدار:

$$\frac{۱۰۰ + ۱۰۰ + ۱۰۰ + ۱۰۰ + ۲۰۰ + ۱۰۰ + ۱۰۰ + ۱۰۰ + ۱۰۰}{۹} = \frac{۱۰۰۰}{۹} \approx ۱۱۱$$

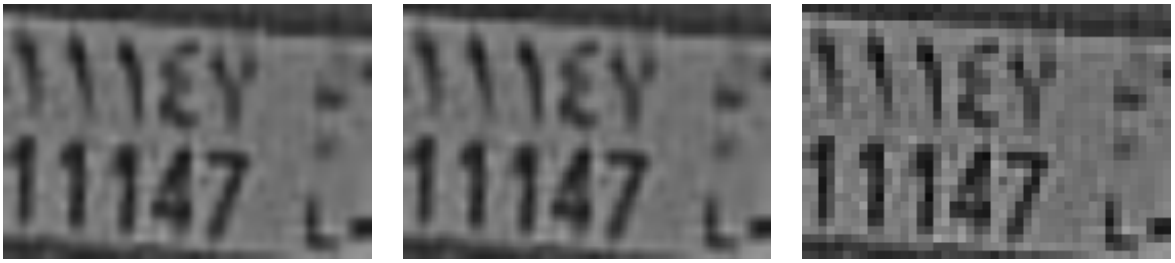
مرکزِ روشن (۲۰۰) به ۱۱۱ پایین آمد — هموار شد. حالا این پنجره یک پیکسل به راست می‌لغزد و همین محاسبه برای مرکزِ بعدی تکرار می‌شود. کلِ تصویر با همین لغزاندن و میانگین‌گیری تار می‌شود.

نکتهٔ کلیدی

نکتهٔ ظریف: مقدارِ خروجی در یک تصویرِ جدید نوشته می‌شود، نه روی همان تصویر. اگر روی همان تصویر بنویسید، پیکسل‌های به‌روزشده روی محاسبهٔ پیکسل‌های بعدی اثر می‌گذارند و نتیجه غلط می‌شود. همیشه به یک بافرِ خروجیِ جدا بنویسید — یک باگِ رایج در پیاده‌سازیِ دستی.

۴ هموارسازی (Smoothing)

۱.۴ تاریِ جعبه‌ای در برابرِ گاوسی — روی پلاکِ واقعی



چپ: اصلی — وسط: جعبه‌ای ۵ × ۵ — راست: گاوسی $\sigma = ۱/۵$

جعبه‌ای به‌طورِ یکسان میانگین می‌گیرد (مصنوعاتِ مربعی)؛ گاوسی به مرکز وزنِ بیشتری می‌دهد (نرم‌تر، طبیعی‌تر).

ریاضیات

کرنلِ گاوسی: $h(i,j) = \frac{۱}{۲\pi\sigma^۲}\, e^{-(i^۲+j^۲)/(۲\sigma^۲)}$

σ میزانِ تاری را کنترل می‌کند؛ بزرگ‌تر = هموارسازیِ بیشتر و کرنلِ پهن‌تر.

گاوسی **تفکیک‌پذیر** (separable) است: یک گاوسیِ دوبعدی = یک تاریِ افقیِ یک‌بعدی سپس یک تاریِ عمودیِ یک‌بعدی → بسیار سریع‌تر.

۲.۴ چرا اصلاً هموار کنیم؟

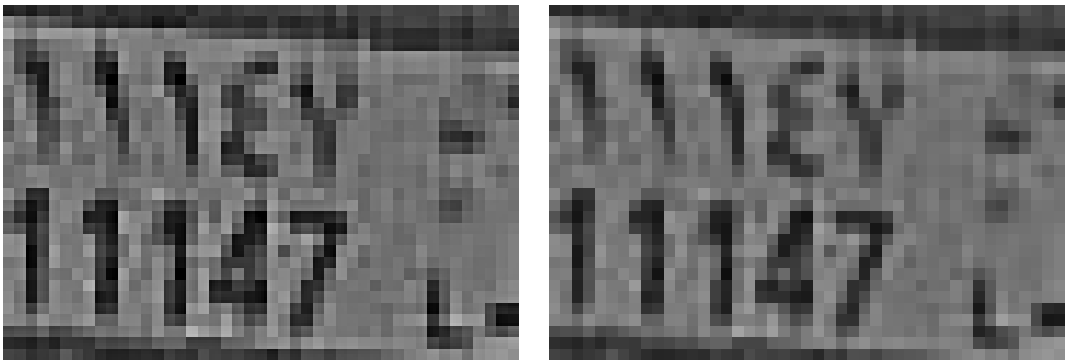
- حذفِ نویز پیش از آستانه/لبه (نویز لبه‌های کاذب می‌سازد — جلسهٔ ۵).
- تاری پیش از کاهشِ اندازه برای پرهیز از آلیاسینگ (نایکوئیستِ جلسهٔ ۱).
- سرکوبِ بافتِ ریز تا ساختارهای بزرگ غالب شوند (یافتنِ صفحه، نه دانه‌بندیِ کاغذ).

هشدار / دام

اما تاری جزئیات را هم نابود می‌کند. زیادی‌اش، ارقام را به هم می‌مالد — یک مبادله که باید تنظیمش کنید.

۵ تیزسازی (Sharpening)

ماسکِ ناتیز (unsharp mask): $\text{تیز} = \text{تصویر} + \lambda(\text{تصویر} - \text{تار})$. جملهٔ $(\text{تصویر} - \text{تار})$ همان جزئیاتِ فرکانس‌بالاست؛ آن را بازمی‌گرداند تا لبه‌ها (ضربه‌قلم‌های ارقام) برجسته شوند.



قبل (چپ) و بعد از تیزسازی با ماسکِ ناتیز (راست)

هشدار / دام

تیزسازی همه‌چیزِ فرکانس‌بالا را تقویت می‌کند — از جمله نویز و بلوک‌های JPEG. تیزسازیِ بیش‌ازحد **هاله** (حلقه‌های روشن/تاریک) دورِ لبه‌ها می‌سازد. اول نویز را حذف کنید، بعد ملایم تیز کنید. هرگز یک تصویرِ نویزی را کورکورانه تیز نکنید.

۶ تمرینِ کلاسیِ وسط‌جلسه (۱۲ دقیقه)

تمرین

یک وصلهٔ ۳ × ۳ و یک کرنل:

$$P = \begin{bmatrix} ۱۰ & ۱۰ & ۱۰ \\ ۱۰ & ۵۰ & ۱۰ \\ ۱۰ & ۱۰ & ۱۰ \end{bmatrix} \qquad K = \begin{bmatrix} ۰ & -۱ & ۰ \\ -۱ & ۵ & -۱ \\ ۰ & -۱ & ۰ \end{bmatrix}$$

(۱) خروجیِ پیکسلِ مرکزی را حساب کنید (ضربِ عنصربه‌عنصر، سپس جمع). (۲) این کرنل با مقدارِ مرکزی چه می‌کند؟ چه نوع فیلتری است؟ (۳) حالا کرنلِ تاریِ جعبه‌ای را به‌جایش بگذارید. خروجیِ مرکز چیست؟

جمع‌بندی: تیزسازی: $۵(۵۰) - ۱۰ - ۱۰ - ۱۰ - ۱۰ = ۲۵۰ - ۴۰ = ۲۱۰$. مرکز (۵۰) به ۲۱۰ پرید — کنتراست تقویت شد. این یک فیلترِ **بالاگذر** است: تفاوتِ مرکز و اطراف را برجسته می‌کند. تاریِ جعبه‌ای: $۱۳۰/۹ \approx ۱۴$ $(۱۰۰۸ + ۵۰)/۹ = ۱۳۰/۹ \approx ۱۴$. مرکزِ روشن میانگین‌گیری و پایین آمد — هموار شد. همان وصله، دو کرنل، دو اثرِ متضاد. کرنل همه‌چیز است.

۷ کاربردهای فیلترِ مکانی

فیلترهای مکانی در تقریباً هر خطِ لولهٔ بینایی‌اند:

- **پیش‌پردازش**: هموارسازیِ ملایم پیش از لبه/آستانه تا نویز، لبه‌های کاذب نسازد.
- **کاهشِ نویز**: گاوسی برای نویزِ گاوسی، میانه برای نمک‌وفلفل، دوطرفه برای حفظِ لبه.
- **تشخیصِ ویژگی**: کرنل‌های گرادیان لبه‌ها را می‌یابند (جلسهٔ ۵)؛ کرنل‌های خاص گوشه و لکه.
- **بهبود**: تیزسازی برای وضوحِ بیشتر (با احتیاط).
- **یادگیریِ عمیق**: لایه‌های اولِ CNN فیلترهای لبه/بافت را می‌آموزند.

یادداشت

یک بینشِ مهم: بیشترِ سیستم‌های بینایی، زنجیره‌ای از فیلترهای مکانی‌اند. کیفیتِ نهایی به انتخابِ درستِ فیلترها و ترتیبشان بستگی دارد. یک فیلترِ غلط در ابتدا (مثلِ تاریِ بیش‌ازحد)، اطلاعاتی را نابود می‌کند که هیچ گامِ بعدی نمی‌تواند بازگرداند.

۱.۷ جدولِ انتخابِ فیلتر

هدف	فیلترِ پیشنهادی
حذفِ نویزِ عمومی، حفظِ کلی	گاوسی
حذفِ نویزِ نمک‌وفلفل	میانه
هموارسازی با حفظِ لبه	دوطرفه
برجسته‌کردنِ لبه‌ها	تیزسازی / گرادیان
آماده‌سازی برای کاهشِ اندازه	گاوسی (ضدِ آلیاسینگ)

۸ بخشِ عمیق

۱.۸ دیدِ فرکانسی: پایین‌گذر در برابرِ بالاگذر

هموارسازی **فرکانس‌های پایین** (تغییراتِ آرام) را نگه می‌دارد و بالا را حذف می‌کند (فیلترِ **پایین‌گذر**). تیزسازی *لبه* **فرکانس‌های بالا** (تغییراتِ سریع) را نگه می‌دارد — فیلترِ **بالاگذر**. جلسهٔ ۶ این را با تبدیلِ فوریه صریح می‌کند؛ امروز شهود است. «تاری = پایین‌گذر، لبه = بالاگذر» عدسی‌ای است که تا آخرِ کورس استفاده می‌کنید.

۲.۸ تصویرِ انتگرالی: تاریِ جعبه‌ای در زمانِ ثابت

یک ترفندِ زیبا: با یک **تصویرِ انتگرالی** (integral image)، که هر پیکسلش مجموعِ همهٔ پیکسل‌های بالا-چپش است)، می‌توان مجموعِ هر مستطیل را با فقط چهار دسترسی حساب کرد — مستقل از اندازهٔ مستطیل. پس یک تاریِ جعبه‌ای با هر اندازه‌ای در زمانِ ثابت به‌ازای هر پیکسل اجرا می‌شود. این پشتِ بسیاری از الگوریتم‌های سریعِ کلاسیک (مثلِ تشخیصِ چهرهٔ Viola-Jones) است.

یادداشت

درسِ کلی: اغلب یک بازنماییِ هوشمندانه (مثلِ تصویرِ انتگرالی) یا یک ویژگیِ ریاضی (مثلِ تفکیک‌پذیری) می‌تواند یک عملیاتِ به‌ظاهر گران را ارزان کند. این تفاوتِ یک پیاده‌سازیِ سادهٔ کند و یک سیستمِ بلادرنگ است.

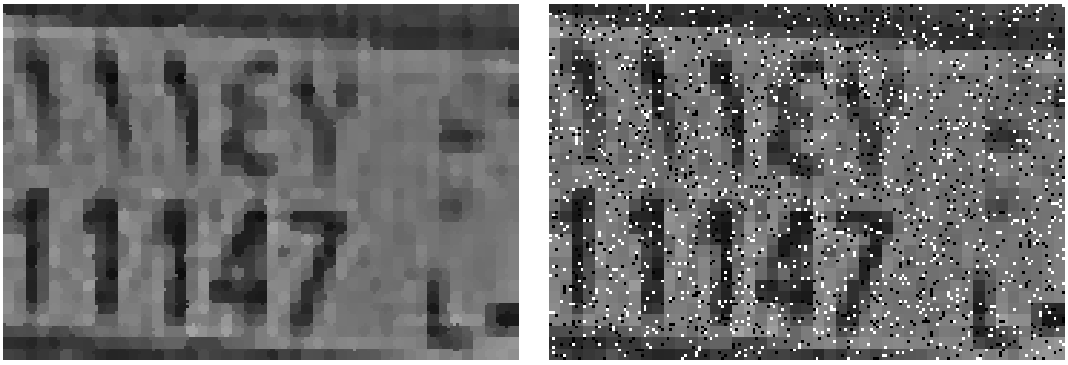
۳.۸ اثراتِ مرزی با جزئیات

روش‌های مختلفِ پُرکردنِ مرز نتایجِ متفاوت می‌دهند: zero (صفر) یک حاشیهٔ تیره می‌سازد؛ replicate آخرین پیکسل را تکرار می‌کند؛ reflect تصویر را آینه می‌کند (معمولاً طبیعی‌ترین). برای تصاویرِ کوچک (مثلِ کراپِ کنتور)، اثرِ مرز نسبتاً بزرگ است، پس انتخابِ روشِ درست مهم‌تر می‌شود.

۴.۸ تفکیک‌پذیری و سرعت

یک کرنلِ دوبعدیِ تفکیک‌پذیر = ضربِ بیرونیِ دو کرنلِ یک‌بعدی؛ آن‌ها را پشتِ‌سرِهم اعمال کنید. یک تاریِ $k \times k$ به‌ازای هر پیکسل k^2 ضرب می‌خواهد؛ تفکیک‌شده، فقط $2k$ — بردی بزرگ برای kهای بزرگ. OpenCV این را برای گاوسی خودکار انجام می‌دهد؛ بدانیدش تا فیلترهای سریع بنویسید.

۵.۸ فیلترِ غیرخطی: میانه (Median)



نویزِ نمک‌وفلفل (چپ) و میانهٔ ۳ × ۳ (راست)

یک فیلترِ **میانه** میانهٔ همسایگی را می‌گیرد — نه جمعِ وزن‌دار. نویزِ نمک‌وفلفل را می‌کشد در حالی که لبه‌ها را تیز نگه می‌دارد (گاوسی آن‌ها را می‌مالید). غیرخطی است ← کانولوشن نیست، اما همان ایدهٔ «پنجره را بلغزان». داستانِ کاملش در جلسهٔ ۴.

۶.۸ فیلترِ خطی در برابرِ غیرخطی

یک تمایزِ بنیادی: فیلترهای **خطی** (تاریِ جعبه‌ای، گاوسی، تیزسازی) با یک جمعِ وزن‌دار کار می‌کنند و همان کانولوشن‌اند؛ خاصیت‌های ریاضیِ خوبی دارند (مثلِ تفکیک‌پذیری). فیلترهای **غیرخطی** (میانه، دوطرفه، بیشینه/کمینه) یک تابعِ غیرخطی روی همسایگی اعمال می‌کنند و کانولوشن نیستند — اما اغلب برای مسائلِ خاص (نویزِ ضربه‌ای، حفظِ لبه) بهترند.

ویژگی	خطی / غیرخطی
جمعِ وزن‌دار	خطی (کانولوشن)
میانه/بیشینه	غیرخطی
تفکیک‌پذیری	فقط خطی‌ها
حفظِ لبه	میانه، دوطرفه (غیرخطی) بهتر

۷.۸ فیلترِ دوطرفه (Bilateral)

تاریِ گاوسی لبه‌ها را می‌مالد؛ فیلترِ **دوطرفه** با هم فاصله و شباهتِ شدت وزن می‌دهد. نتیجه: نواحیِ تخت هموار می‌شوند، لبه‌ها تیز می‌مانند — عالی پیش از آستانه‌گذاریِ یک سند. کندتر، اما اغلب هموارسازِ درست برای DocScan: cv2.bilateralFilter.

۹ فیلترها در کد

```
box    = cv2.blur(img, (5, 5))
gauss  = cv2.GaussianBlur(img, (7, 7), (5.1
median = cv2.medianBlur(img, 3)
k = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp  = cv2.filter2D(img, -1, k)          # apply any custom kernel
```

cv2.filter2D با هر کرنلی که بدهید کانولوشن می‌کند — ابزارِ عمومی. برای فهمِ عمیق، یک‌بار کانولوشن را از صفر با NumPy پیاده کنید:

```
def convolve3x3(img, k):
    h, w = img.shape
    pad = np.pad(img.astype(np.float32), 1, mode='reflect')  # handle borders
    out = np.zeros_like(img, dtype=np.float32)
    for y in range(h):
        for x in range(w):
            region = pad[y:y+3, x:x+3]
            out[y, x] = np.sum(region * k)          # multiply + sum
    return np.clip(out, 0, 255).astype(np.uint8)
```

این نسخهٔ آموزشی (با حلقه) کند است اما مفهوم را روشن می‌کند. در عمل cv2.filter2D همین را هزاران برابر سریع‌تر (برداری‌شده و بهینه) انجام می‌دهد. نکته: با np.pad مرزها را مدیریت کردیم و به یک بافرِ خروجیِ جدا نوشتیم.

۱.۹ پارامترها و دام‌ها

- **اندازهٔ کرنل**: باید فرد باشد (۳, ۵, ۷, . . .) تا مرکزِ مشخص داشته باشد.
- **گاوسی** σ: اگر ۰ بدهید، OpenCV از اندازهٔ کرنل محاسبه‌اش می‌کند؛ برعکس هم.
- **عمقِ خروجی**: filter2D با -1 همان نوعِ ورودی؛ اما کرنل‌های لبه مقادیرِ منفی می‌دهند که در uint8 بریده می‌شوند — به float بروید.
- **مرز**: پیش‌فرضِ reflect معمولاً بهتر از zero است (لبه‌های تیرهٔ مصنوعی نمی‌سازد).

هشدار / دام

دامِ رایج: اعمالِ یک کرنلِ لبه (با مقادیرِ منفی) روی تصویرِ uint8 — نتایجِ منفی به صفر بریده می‌شوند و نیمی از لبه‌ها ناپدید می‌شوند. برای فیلترهایی که خروجیِ منفی دارند، ابتدا به float32 تبدیل کنید و در آخر نرمال‌سازی/قدرمطلق بگیرید.

۱۰ باغِ کرنل‌ها و طراحیِ آن‌ها

شما کرنل‌ها را حفظ نمی‌کنید — می‌فهمید «میانگین» در برابرِ «تفاضل» چه می‌کند و بقیه را طراحی می‌کنید:

کرنل	اثر
همه $1/n$	تاریِ جعبه‌ای (میانگین)
وزن‌های گاوسی	تاریِ نرم
مرکز ۵، اطراف -1	تیزسازی (بالاگذر)
سطرهای $[-1, 0, 1]$	گرادیانِ افقی (لبه، جلسهٔ ۵)
هویت + (هویت − تار)	ماسکِ ناتیز

منطقِ طراحی: اگر می‌خواهید هموار کنید، کرنلی بسازید که مجموعِ مثبت داشته و به مرکز وزن دهد. اگر می‌خواهید تغییرات (لبه) را برجسته کنید، کرنلی با مجموعِ صفر بسازید که تفاوتِ همسایه‌ها را می‌گیرد. این دو الگو، پشتِ تقریباً همهٔ فیلترهای خطی‌اند.

یادداشت

یک کرنلِ جالب: **برجسته‌سازی** (emboss) با $\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$ که یک اثرِ سه‌بعدیِ نور از یک جهت می‌سازد. این نشان می‌دهد چطور چیدمانِ نامتقارنِ اعداد، جهت‌داری ایجاد می‌کند — همان ایده که در تشخیصِ لبهٔ جهت‌دارِ جلسهٔ ۵ به کار می‌آید.

۱۱ کانولوشن در برابرِ همبستگی

یک نکتهٔ نظری که در عمل مهم می‌شود: **کانولوشن** کرنل را پیش از اعمال ۱۸۰ درجه می‌چرخاند، اما **همبستگی** (correlation) نه. برای کرنل‌های متقارن (مثلِ گاوسی یا تاریِ جعبه‌ای) این دو نتیجهٔ یکسان می‌دهند، پس اغلب تفاوتی حس نمی‌کنید. اما برای کرنل‌های نامتقارن (مثلِ تشخیصِ لبهٔ جهت‌دار)، اهمیت دارد. `cv2.filter2D` در واقع همبستگی انجام می‌دهد؛ اگر کانولوشنِ واقعی می‌خواهید، کرنل را اول بچرخانید.

نکتهٔ کلیدی

در یادگیریِ عمیق، آنچه «لایهٔ کانولوشن» نامیده می‌شود در واقع همبستگی است (کرنل چرخانده نمی‌شود). چون شبکه کرنل را از داده می‌آموزد، چرخشِ آن اهمیتی ندارد — شبکه هر طور که لازم باشد یاد می‌گیرد. این یک نمونه از جاهایی است که اصطلاحاتِ ریاضی و عملی کمی متفاوت‌اند.

۱۲ این جلسه در پروژهٔ نهایی

در DocScan، هموارسازی نقشِ حیاتی دارد اما ظریف است: یک تاریِ ملایم پیش از آستانه‌گذاری، نویز را حذف می‌کند و آستانهٔ تمیزتری می‌دهد؛ اما تاریِ زیاد ارقام را به هم می‌مالد و جداسازیِ رقم‌ها (جلسهٔ ۹) را خراب می‌کند. برای عکس‌های موبایلیِ نویزی، فیلترِ دوطرفه یا میانه اغلب بهترند از گاوسی، چون لبه‌های ارقام را حفظ می‌کنند.

یادداشت

یک قاعدهٔ عملی: کوچک‌ترین تاری‌ای را که نویز را کافی حذف کند به کار ببرید. هر واحدِ تاریِ اضافه، جزئیاتی است که برای خواندنِ رقم از دست می‌دهید. این تعادل را روی چند تصویرِ سخت تنظیم کنید.

۱۳ جمع‌بندی

- توضیحِ کانولوشن و اینکه چطور یک کرنل عملیات را تعریف می‌کند.
- هموارسازی با جعبه/گاوسی؛ تیزسازی با ماسکِ ناتیز؛ دانستنِ مبادله‌ها.
- مدیریتِ مرز و نرمال‌سازی؛ بهره‌گیری از تفکیک‌پذیری برای سرعت.
- دیدِ پایین‌گذر/بالاگذر و پیوند به CNNها.

تمرین

تحویلیِ این هفته (پیش از جلسهٔ ۴): روی تصاویرِ واقعیِ کنتور/پلاک: (۱) یک کانولوشنِ ۳×۳ را از صفر (NumPy) پیاده و با cv2.filter2D تطبیق دهید. (۲) تاریِ جعبه‌ای و گاوسی را در چند اندازه مقایسه کنید؛ ازدست‌رفتنِ جزئیات در برابرِ حذفِ نویز را بحث کنید. (۳) یک پلاک را با ماسکِ ناتیز تیز کنید؛ بیابید کجا شروع به ساختنِ هاله می‌کند. (۴) یادداشت: چه تاری‌ای به آستانهٔ DocScan شما کمک می‌کند و چه تاری‌ای ارقام را بیش‌ازحد می‌مالد؟

واژه‌نامهٔ کوتاه

اصطلاح	معنی
Convolution	لغزاندنِ یک کرنل و جمعِ وزن‌دارِ همسایگی
Kernel / Filter	ماتریسِ کوچکی که عملیات را تعریف می‌کند
Low-pass / High-pass	نگه‌داشتنِ فرکانسِ پایین (تاری) / بالا (لبه)
Separable	کرنلِ دوبعدی = دو کرنلِ یک‌بعدی (سریع‌تر)
Unsharp mask	تیزسازی با افزودنِ جزئیاتِ فرکانس‌بالا
Median / Bilateral	فیلترِ غیرخطیِ حفظ‌کنندهٔ لبه

کرنل، الگوریتم است. تاری، تیزی، لبه — همه یک عملیات با اعدادِ متفاوت.